

From analyst bottleneck to self-service analytics

How I identified the problem, framed the product, led validation, and drove it to adoption

100+

internal users

~20%

fewer ad-hoc analytics tickets

~3 mo.

from launch to measured impact

Context: where the product problem lived

AiDash uses satellite imagery, geospatial analytics and AI to help utilities monitor infrastructure and prioritize maintenance across large physical networks.

USER

Internal business users

- Needed answers quickly for recurring operational and customer questions
- Questions changed often and did not map cleanly to fixed dashboards
- Most requests were time-sensitive, but many were not analytically complex

CURRENT WORKFLOW

Analysts became the bottleneck

- Repeated schema lookup and query writing
- Interrupt-driven work reduced focus on higher-value analysis
- Turnaround depended on analyst availability

MY PRODUCT MANDATE

Move routine work to self-service without sacrificing trust

- Frame the product problem and define the bounded first scope
- Own business schema / semantics and clear acceptance criteria
- Co-own guardrails, evaluation and transition with Engineering
- Drive UAT, rollout, adoption and outcome measurement

Self-service first • Human judgment when needed

Discovery: the repeated work was hiding in plain sight

I started by categorizing incoming ad-hoc analytics requests rather than jumping to an AI solution.

≈80%

of sampled ad-hoc requests were straightforward, repetitive query runs

Important: this was not 80% of all analytics work.

The recurring pattern



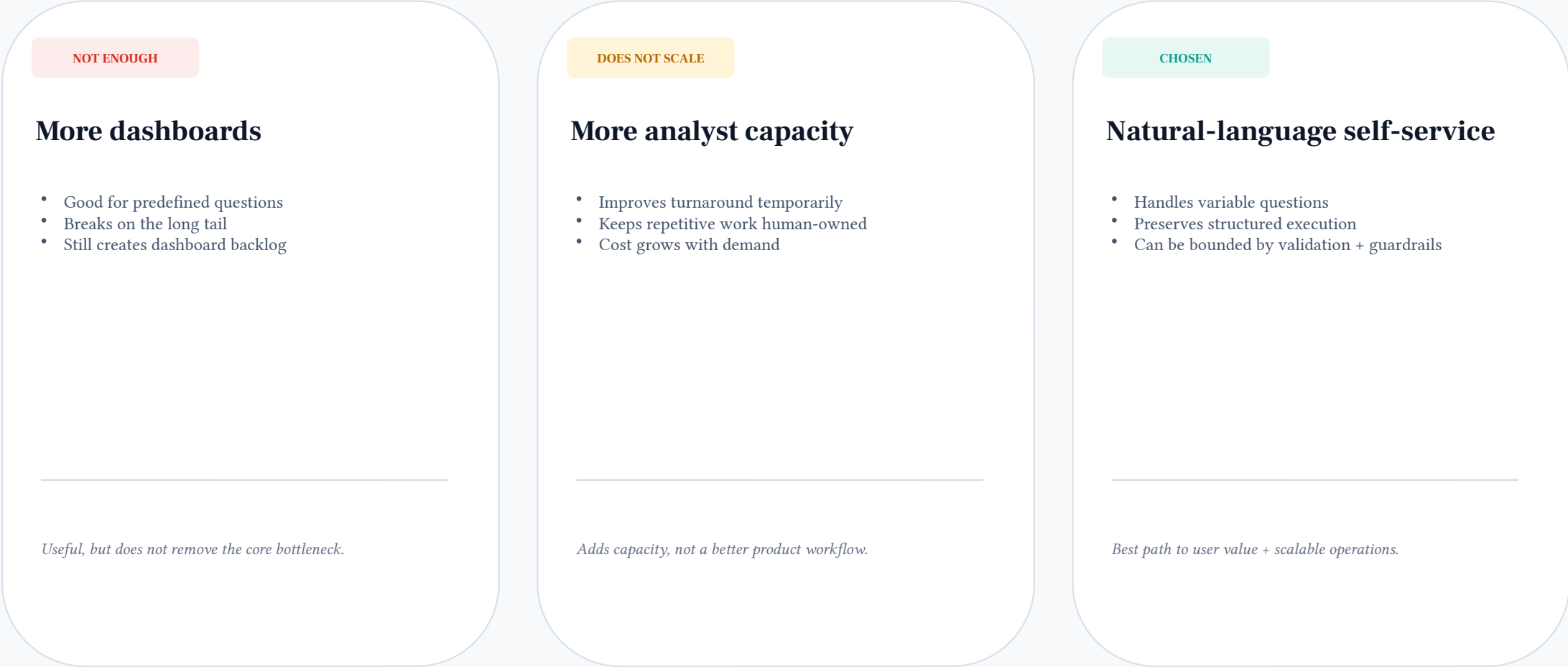
PRODUCT INSIGHT

Automate the repeatable path — not the judgment-heavy analysis.

That became the boundary for the first release.

Product decision: why self-service AI, not “just another dashboard”

The long tail of questions changed too often for a fixed interface, but the repeatable workflow was structured enough to constrain.



Success definition: shift routine questions away from analyst queues — while keeping execution safe, bounded and explainable.

First release: narrow enough to trust, useful enough to change behavior

I deliberately constrained the initial scope to the highest-frequency, lowest-risk analytical workflows.

IN SCOPE

-  **Natural-language questions** Users ask in their own words
-  **Relevant schema retrieval** Only context needed for the question
-  **SQL generation + validation** Intent → query → safe execution
-  **Clarification / bounded retry** Explicit branches for ambiguity / failure
-  **Read-only execution** No source-data mutation

DELIBERATELY OUT OF SCOPE

-  **Complex analytical judgment** Kept with analysts
-  **Write-back actions** No mutation of source data
-  **“Answer anything” behavior** Supported workflows over broad coverage
-  **Broad autonomy from day one** Expand only after quality + trust are proven

Product principle: earn trust before expanding autonomy.

My role: clear product ownership, clear engineering boundaries

I led the product decisions and release readiness while co-owning AI quality controls with Engineering.

PM OWNED

- Problem framing
- Business schema / semantics
- Rollout & adoption

SHARED / CO-OWNED

- Guardrails & safety
- Eval suite / golden dataset
- Prototype transition

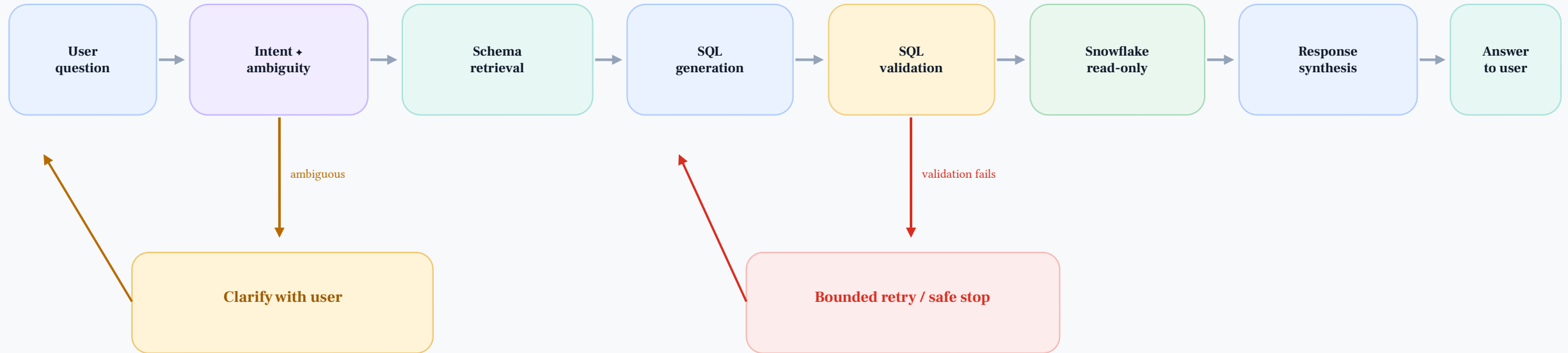
ENGINEERING OWNED

- Production architecture
- Data platform / Snowflake
- Reliability & observability

PM principle: own the problem, product behavior, acceptance and adoption — without taking credit for engineering implementation.

Product workflow: simple conversation, explicit failure handling

The user sees a natural-language experience; the product routes ambiguity and execution failures through controlled branches.



Key product behavior: ambiguity is clarified before retrieval; invalid SQL retries back to generation; successful results are synthesized into a user-facing answer.

Three product decisions that shaped the experience

The hard part was not choosing a model; it was deciding where the product should be flexible and where it should be constrained.

01

Coverage vs. trust

Should it answer everything?

No. Start with repeatable workflows and clarify ambiguity rather than guessing.

02

Speed vs. release confidence

How fast should we expand?

Use the eval suite and acceptance criteria as release gates; broaden scope only after quality is demonstrated.

03

Autonomy vs. control

Let the LLM execute freely?

No. Restrict context, validate SQL, keep execution read-only and preserve safe failure behavior.

PM guardrail lens: define where AI may act, when it must clarify, what must be validated, and when the workflow must stop safely.

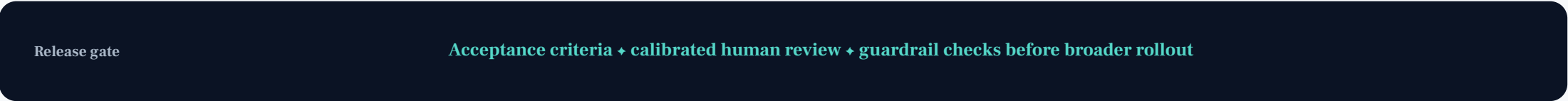
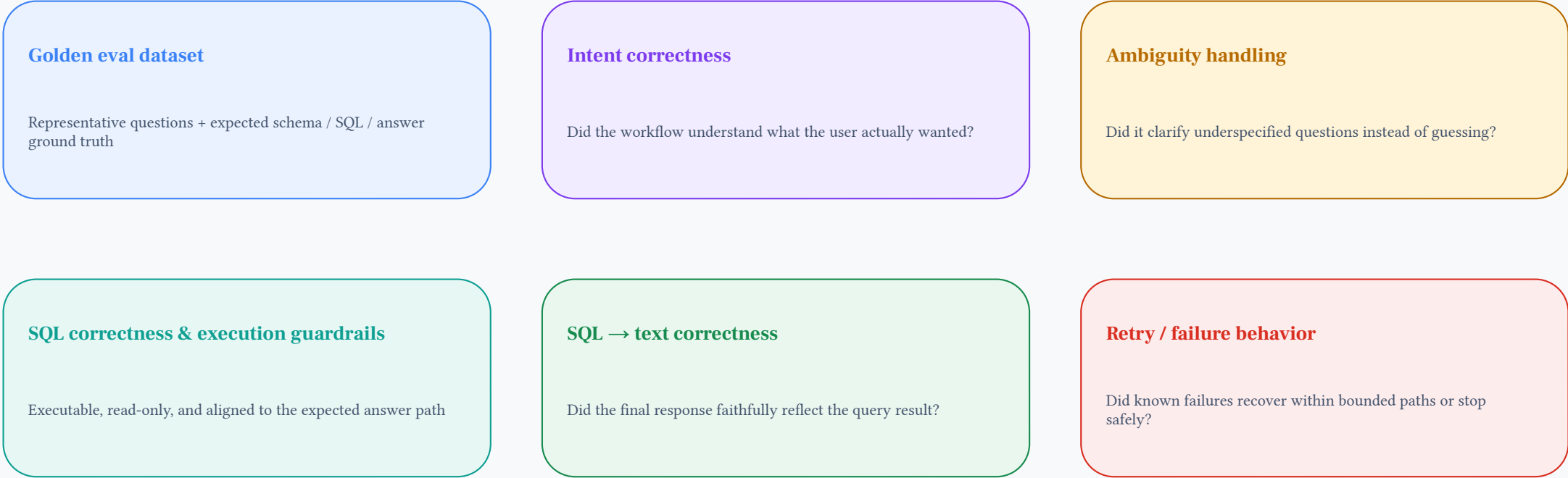
Technical architecture: reasoning where it helps, deterministic controls where trust matters

I led the product behavior and quality gates; Engineering owned the production architecture and reliability layer.



Evaluation, guardrails & release readiness: AI quality was a product requirement

We used a golden eval dataset, explicit guardrails, acceptance criteria and human-review calibration to decide whether the workflow was ready to expand.



How I got the work done: product leadership ♦ technical fluency

The role required translating between user problems, AI behavior and engineering constraints without relying on formal authority.

PRODUCT

- Discovery & problem framing
- Scope / prioritization
- Acceptance criteria
- UAT, rollout & metrics

AI PRODUCT

- Intent / ambiguity design
- Guardrails & safety
- Golden eval dataset
- Failure / retry behavior

TECHNICAL FLUENCY

- SQL / Python
- RAG + LangGraph
- Snowflake / schemas
- AI failure analysis

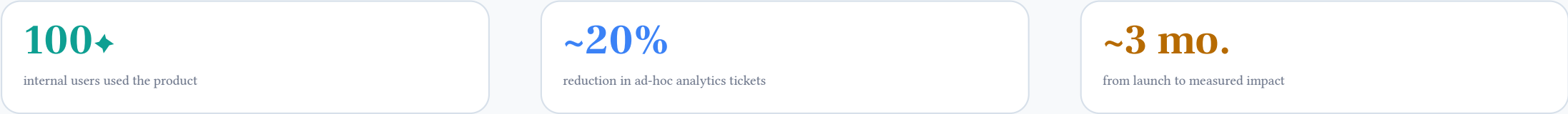
INTERPERSONAL

- User / stakeholder discovery
- Engineering partnership
- Influence without authority
- Clear ownership boundaries

Operating principle: be firm on the problem, outcome and acceptance bar; be flexible on implementation.

Launch and impact: the product changed the operating model

Within ~3 months, routine questions moved away from analyst queues and into self-service.



What changed operationally

- 1

Business users

Faster answers for routine questions
- 2

Analysts

Less repeated query execution; more time for higher-value analysis
- 3

Product / Engineering

Reusable foundation for conversational analytics

The metric that mattered most

Not “number of chats.”

Did the product actually remove repetitive work from the analyst queue?

What this project taught me about AI product management

The technical system mattered — but the product decisions around workflow, evaluation, trust and adoption mattered more.



What I'd do differently	Instrument the product journey earlier Separate AI-quality failures from UX / adoption friction.	Define explicit expansion gates Use quality, usage and trust thresholds before broadening workflows or autonomy.
-------------------------	--	--

Thank you

Questions or follow-up?

Please feel free to reach out if you have any questions.

Nitish Ranjan

ranjannitish30@gmail.com • +91-9 911 651 252
[linkedin.com/in/ranjan-nitish](https://www.linkedin.com/in/ranjan-nitish)

